

## ESQUEMAS

**-foldr:** esquema general de recursión.

-Dada una función de incorporación de elementos, un valor básico y una lista, describe el resultado de procesar e incorporar todos los elementos de esa lista

**foldr :: (a->b->b) -> b -> [a] -> b**

foldr f z [] = z

foldr f z (x:xs) = f x (foldr f z xs)

**-map:** aplica una transformación a cada elemento de una lista, devolviendo una nueva lista con los resultados

-El esquema de map (asociación), transforma listas elemento a elemento, según el parámetro dado

**map :: (a->b) -> [a] -> [b]**

map f [] = []

map f (x:xs) = f x : map f xs

**-filter:** El esquema de filter elimina elementos que no cumplen la condición dada

**filter :: (a->Bool) -> [a] -> [a]**

filter p [] = []

filter p (x:xs) = if p x then x : filter p xs else filter p xs

**-recr:** Es un esquema similar al de foldr

-Para no mezclarlos, el orden de los parámetros es inverso

-La función parámetro ahora tiene un parámetro más

**recr :: b -> (a->[a]->b->b) -> [a] -> b**

recr z f [] = z

recr z f (x:xs) = f x xs (recr z f xs)

---

## OTROS:

-otro caso de foldr

**foldr1 :: (a -> a -> a) -> [a] -> a**

foldr1 f [] = error "lista vacía" --el caso base como no se pasa tiene que ser un elemento de la lista

foldr1 f [x] = x --caso base es cuando la lista tiene un solo elemento

foldr1 f (x:xs) = f x (foldr1 f xs)

**-zipWith:** Funciona igual que zip, pero te permite aplicar una función específica a cada par de elementos en lugar de simplemente agruparlos en una tupla.

-sirve para combinar dos listas elemento a elemento usando una función que vos le das.

```
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith f xs [] = []
zipWith f [] ys = []
zipWith f (x:xs) (y:ys) = f x y : (zipWith f xs ys)
```

**-scanr**: devuelve todos los estados intermedios desde la derecha

```
scanr :: (a -> b -> b) -> b -> [a] -> [b]
scanr f b [] = [b]
scanr f b (x:xs) =
  let rec = scanr f b xs
  in f x (head rec) : rec
```

---

**-sum**: suma todos los elementos

```
sum :: [Int] -> Int
sum foldr (+) 0
```

**-length**: devuelve la longitud de la lista

```
length :: [a] -> Int
length = foldr (\_ b -> 1 + b) 0
```

~...

```
map :: (a -> b) -> [a] -> [b]
map f = foldr (\x b -> f x : b) []
```

~...

```
filter :: (a -> Bool) -> [a] -> [a]
filter f = foldr (\x b -> if f x then x : b else b) []
```

**-find**: Devuelve el primer elemento que cumple (de izquierda a derecha).

```
find :: (a -> Bool) -> [a] -> Maybe a
find f = foldr (\x b -> if f x then Just x else b) Nothing
```

**-any**: devuelve si existe alguno que cumpla con la función dada

```
any :: (a -> Bool) -> [a] -> Bool
any f = foldr (\x b -> f x || b) False
```

**-all:** devuelve si todos cumplen con la condición dada

**all :: (a -> Bool) -> [a] -> Bool**

all f = foldr (\x b -> f x && b) True

**-countBy:** cuenta cuantos cumplen la condición dada

**countBy :: (a -> Bool) -> [a] -> Int**

countBy f = foldr (\x b -> if f x then 1 + b else b) 0

**-partition:** separa en dos listas los que no cumplen y los que cumplen

**partition :: (a -> Bool) -> [a] -> ([a], [a])**

partition p = foldr (\x (ts, fs) -> if p x then (x:ts, fs) else (ts, x:fs)) ([], [])

**-takeWhile:** Toma elementos mientras cumplan.

**takeWhile :: (a -> Bool) -> [a] -> [a]**

takeWhile p = foldr (\x b -> if p x then x : b else []) []

**-take:** Toma los primeros n elementos.

**take :: Int -> [a] -> [a]**

take = flip (foldr ctk (\n -> []))

where ctk x h = \n -> if n==0 then [] else x : h (n-1)

**-drop:** Saca los primeros n elementos.

**drop :: Int -> [a] -> [a]**

drop n xs = snd (foldr (\x (i, acc) ->

if i < n then (i+1, acc)

else (i, x:acc)) (0, []) xs) —otra manera fea de hacerlo

**-crossWith:** describe el resultado de aplicar una función a cada elemento del producto cartesiano de las dos listas dadas.

-Por ejemplo

crossWith (+) [1,2,3] [10,20] da como resultado [11,21,12,22,13,23].

**crossWith :: (a -> b -> c) -> [a] -> [b] -> [c]**

crossWith f xs ys = foldr (\x b -> map (f x) ys ++ b) [] xs

-Sacar el **ancho de un arbol** = zipPerLevel de dicho arbol y a esa lista buscar el máximo:

-El máximo se busca sobre las longitudes de los niveles (zipPerLevel)

Ej:

**foldT :: b -> (a -> b -> b -> b) -> Tree a -> b**

foldT c f EmptyT = c

foldT c f (NodeT x t1 t2) =

f x (foldT c f t1) (foldT c f t2)

**zipPerLevel :: [[a]] -> [[a]] -> [[a]]**

zipPerLevel [ ] ys = ys

zipPerLevel xs [ ] = xs

zipPerLevel (x:xs) (y:ys) = (x++y) : zipPerLevel xs ys

**nodosPorNivel :: Tree a -> [[a]]**

nodosPorNivel = foldT [ ] (\a izq der -> [a] : zipPerLevel izq der)

**ancho :: Tree a -> Int**

ancho = maximum . map length . nodosPorNivel